Informe Parcial del Firmware iGate APRS-IS

Sistema de Rastreo en Tiempo Real para Operaciones de Rescate Urbano

Grupo 2
Taller Integrador
Instituto Tecnologico de Costa Rica

Resumen

Los grupos de Rescate Urbano que operan en zonas montañosas enfrentan un problema critico: una vez que los rescatistas ingresan al terreno, el coordinador pierde visibilidad de su posicion. Este informe parcial documenta el desarrollo de un iGate APRS-IS propio, basado en ESP32 y un transceptor LoRa SX1276, como componente central de un sistema de rastreo en tiempo real que no depende de infraestructura celular. El iGate se instala en el puesto de comando, recibe por radio LoRa las posiciones de los rescatistas y las publica en la red APRS-IS a traves de WiFi, permitiendo al coordinador visualizar el despliegue del equipo desde cualquier dispositivo con acceso a aprs.fi. El desarrollo se organiza alrededor de una maquina de estados finita con los estados `START`, `LISTEN`, `DECODE`, `UPLINK`, `BEACON_TX` y `ERROR`. El trabajo cubre la arquitectura del sistema, la configuracion de perifericos, el pseudocodigo de alto nivel, la definicion de tramas de datos, el cronograma de ejecucion y un presupuesto estimado del prototipo.

Index Terms

APRS-IS, ESP32, Firmware embebido, LoRa, Maquina de estados, PlatformIO, Rastreo, Rescate urbano.

I. Introduccion

Los grupos de Rescate Urbano realizan operaciones en zonas montañosas y boscosas donde la cobertura celular es inexistente o intermitente. Durante estas operaciones, el coordinador en el puesto de comando pierde visibilidad de la posicion del equipo una vez que los rescatistas ingresan al terreno, lo que dificulta la toma de decisiones y la coordinacion ante emergencias.

El sistema propuesto resuelve este problema mediante una red LoRa-APRS de bajo costo. Cada rescatista porta un modulo rastreador (tracker) que transmite su posicion por radio LoRa. El iGate, instalado en el puesto de comando donde existe acceso a WiFi o un hotspot movil, recibe estas transmisiones y las reinyecta a la red APRS-IS mediante TCP. El coordinador puede entonces visualizar el despliegue completo del equipo en tiempo real desde cualquier dispositivo con acceso a `aprs.fi`, sin depender de infraestructura celular en la zona de operacion.

La tecnologia LoRa es adecuada para este escenario por su alcance tipico de 2 a 5 km en terreno montañoso con los parametros configurados (SF=12, BW=125 kHz), su bajo consumo de energia y su operacion en la banda de 433 MHz, libre de licencia para uso experimental en Costa Rica segun el Plan Nacional de Atribucion de Frecuencias (PNAF).

La Fig. 1 ilustra el escenario de despliegue propuesto para Rescate Urbano.



Figura 1: Escenario de rastreo LoRa-APRS para operaciones de Rescate Urbano. Los rescatistas portan trackers que transmiten por radio LoRa. El iGate en el puesto de comando recibe las posiciones y las publica en tiempo real via TCP/IP.

El firmware implementado toma como base una arquitectura modular: `main.cpp` coordina la maquina de estados, `wifi_manager.cpp` gestiona la conexion a la red, `lora_manager.cpp` inicializa y opera el enlace LoRa, `aprs_parser.cpp` valida y reformatea las tramas, `aprs_is_client.cpp` administra el socket TCP y `beacon.cpp` construye la posicion periodica del iGate. Esta separacion facilita las pruebas unitarias por bloque, la depuracion y la futura extension del sistema.

II. Objetivo del entregable parcial

El alcance del informe parcial es documentar el estado actual del firmware y justificar las decisiones de diseno en el contexto de la aplicacion de rescate. En particular, este documento se enfoca en:

- describir la arquitectura funcional del iGate y los perifericos utilizados;
- presentar la maquina de estados y el flujo de ejecucion del firmware;
- explicar el pseudocodigo general que resume el comportamiento de los modulos;
- definir las tramas de datos intercambiadas por radio y por red;
- incluir un cronograma de ejecucion y un presupuesto estimado del prototipo.

III. Arquitectura propuesta del sistema

III-A. Vista general

La arquitectura del sistema sigue una cadena de procesamiento que inicia en el tracker del rescatista, continua con la recepcion por radio LoRa en el iGate ubicado en el puesto de comando, y termina en el reenvio a un servidor APRS-IS a traves de TCP. El coordinador accede al mapa de posiciones en tiempo real desde `aprs.fi` o desde el servidor local de la clase. La Fig. 2 resume el flujo funcional y el papel de cada capa.

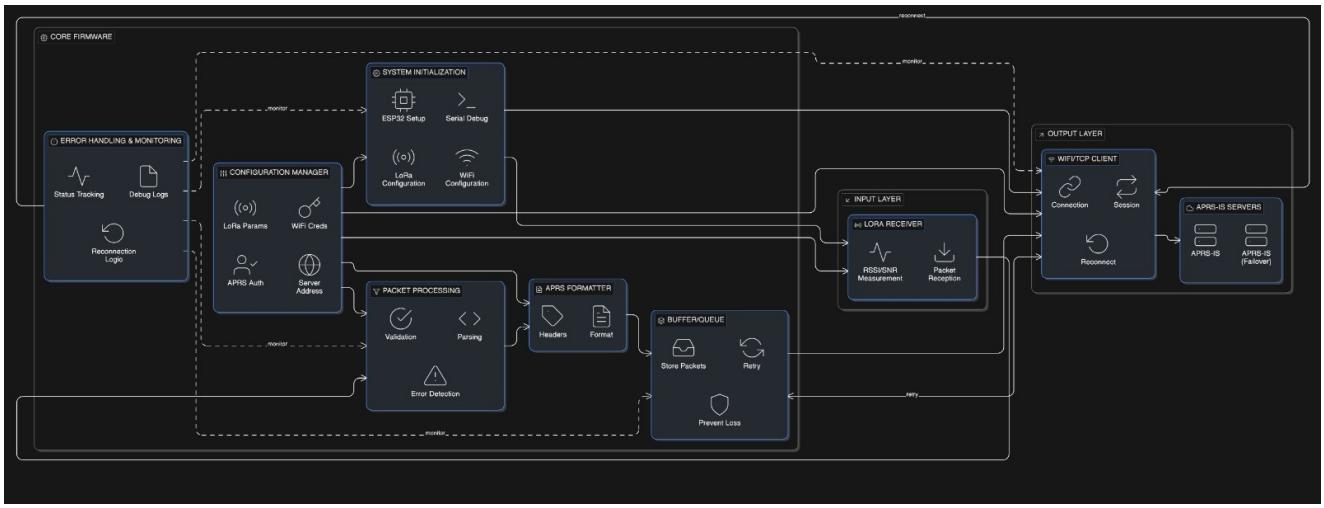


Figura 2: Diagrama general de bloques del ecosistema propuesto para el iGate APRS-IS.

En el interior del firmware, la inicializacion del sistema habilita depuracion serial, pantalla OLED opcional, pila WiFi y parametros de radio LoRa. Posteriormente, el lazo principal mantiene la escucha de paquetes, valida la integridad de las tramas recibidas y, cuando una trama es aceptada, la transforma al formato requerido por APRS-IS. De forma paralela, un temporizador genera el beacon del iGate y un estado de error centraliza la recuperacion ante fallos.

III-B. Modulos de software

La Tabla I resume la responsabilidad de los archivos principales del firmware propio.

Tabla I: Modulos principales del firmware y su funcion.

Modulo	Responsabilidad
`main.cpp`	Coordina la maquina de estados, las transiciones y los temporizadores globales.
`wifi_manager.cpp`	Inicia la interfaz WiFi en modo estacion, espera la asociacion y expone el estado de conectividad.
`lora_manager.cpp`	Configura SPI, inicializa el transceptor LoRa y recibe paquetes junto con sus metricas RSSI/SNR.
`aprs_parser.cpp`	Valida la sintaxis del paquete LoRa-APRS y lo reformatea a trama APRS-IS.
`aprs_is_client.cpp`	Abre el socket TCP, envia el login APRS-IS y transmite lineas terminadas en `\r\n`.
`beacon.cpp`	Convierte coordenadas decimales a formato APRS y genera el beacon periodico del iGate.
`display.cpp`	Presenta informacion de arranque, estados y eventos en la OLED SSD1306 cuando esta habilitada.

III-C. Perifericos y protocolos

La seleccion de protocolos esta determinada por los componentes fisicos del prototipo. El modulo LoRa SX1276 se controla por **SPI**, la pantalla SSD1306 usa **I2C**, la depuracion se hace por **UART** a 115200 bps y el reenvio a APRS-IS se hace por **WiFi/TCP**. La Tabla II sintetiza estas decisiones.

Tabla II: Perifericos, protocolo y justificacion de uso.

Periferico	Protocolo	Justificacion
SX1276	SPI + GPIO	El transceptor requiere reloj, datos y seleccion de chip para configurar frecuencia, SF, BW, CR y lectura de paquetes.
OLED SSD1306	I2C	Permite visualizar estado del sistema con solo dos lineas de bus, reduciendo pines y complejidad de cableado.
Depuracion serial	UART	Facilita validacion de arranque, pruebas de conexion y monitoreo de eventos en clase.
APRS-IS	TCP sobre WiFi	El protocolo APRS-IS utiliza sesiones orientadas a conexion y autenticacion por login textual.

El mapeo de pines fijado en `config.h` corresponde al hardware TTGO LoRa32 T3 v1.6.1, verificado contra el esquematico oficial de la tarjeta. Para el modulo LoRa se usan `SCK=5`, `MISO=19`, `MOSI=27`, `SS=18`, `RST=23` y `DIO0=26`. Para la OLED se definen `SDA=21` y `SCL=22`. A nivel de radio, el firmware fija `433.775 MHz`, `BW=125 kHz`, `SF=12`, `CR=4/5`, `SyncWord=0x12` y CRC habilitado, que corresponden al esquema de operacion previsto para LoRa-APRS.

III-D. Diagrama electrico del prototipo

Con el fin de complementar la justificacion de perifericos, se anade el esquema electrico del hardware base utilizado para el desarrollo. La Fig. 3 agrupa los bloques principales del equipo: conectores laterales, nucleo ESP32, subsistema de radio LoRa SX1276 y etapa de alimentacion/regulacion. La Fig. 4 resume los bloques auxiliares integrados en la tarjeta, incluyendo la pantalla OLED, las lineas de LED y el slot MicroSD.

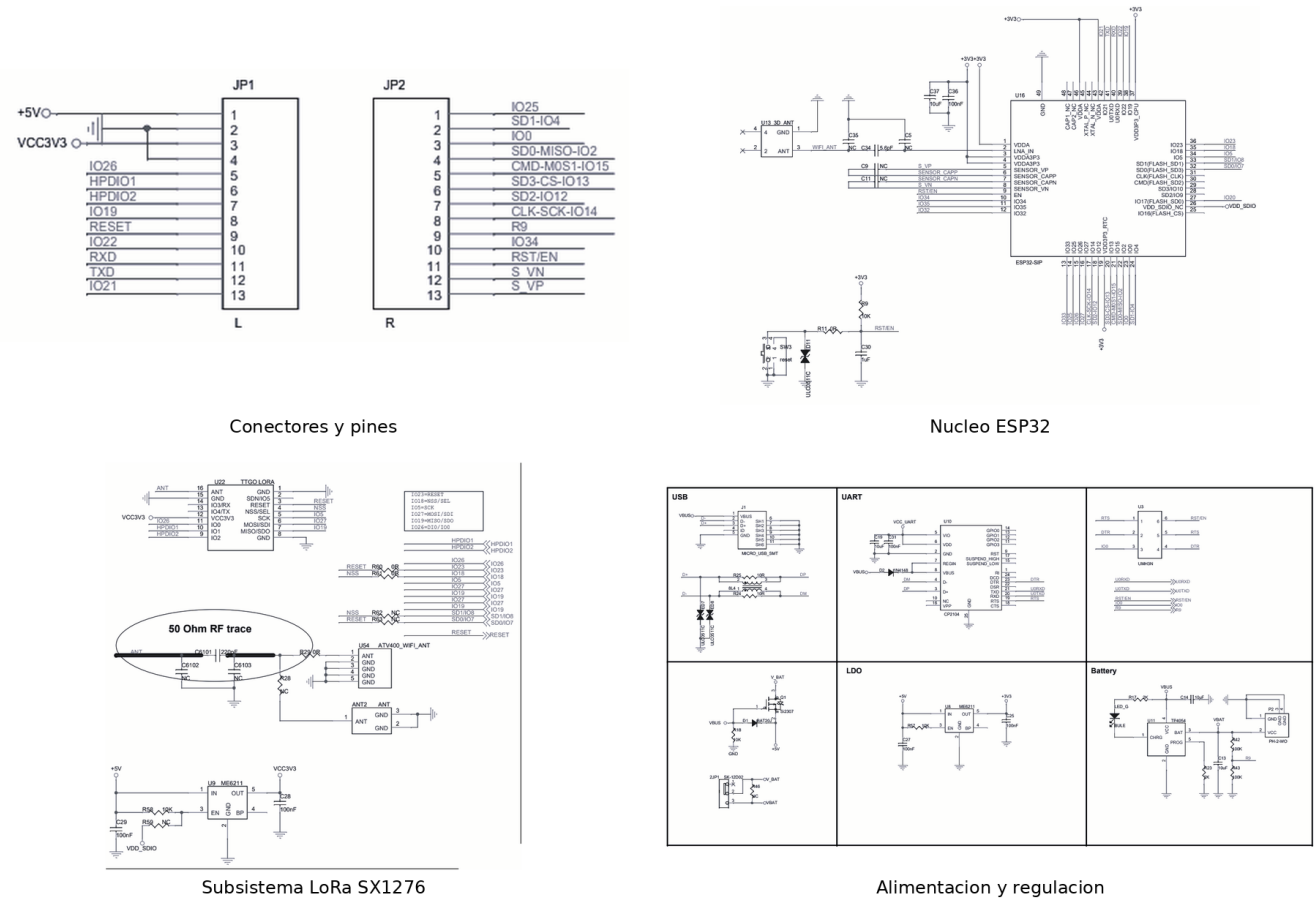


Figura 3: Esquema electrico resumido del hardware base del iGate: conectores, nucleo ESP32, radio LoRa SX1276 y alimentacion.

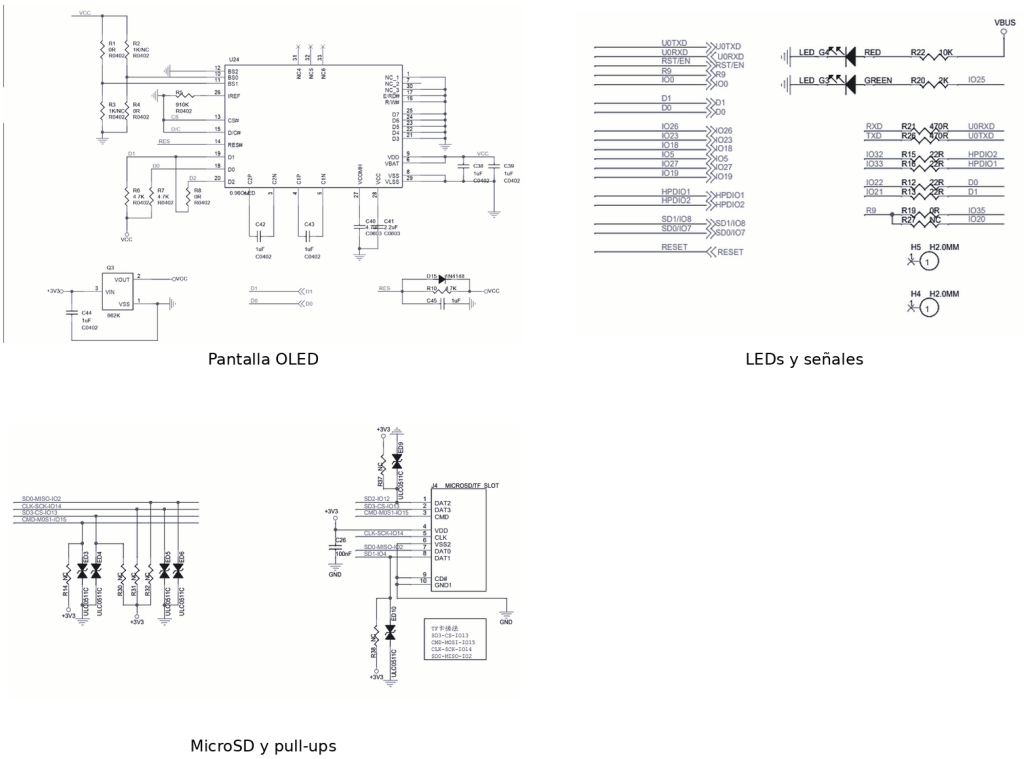


Figura 4: Bloques electricos auxiliares del prototipo: OLED, LEDs de estado y slot MicroSD con sus redes de soporte.

Desde el punto de vista funcional, el esquema electrico respalda las decisiones de protocolo descritas en la Tabla II. El transceptor LoRa se enlaza con el ESP32 mediante SPI y una linea de interrupcion digital (DIO0); la pantalla OLED utiliza un bus I^2C de baja complejidad; y la alimentacion integra regulacion local para sostener la operacion del microcontrolador, el radio y los perifericos de interfaz.

IV. Maquina de estados finita

El control del iGate se organiza como una maquina de estados finita (FSM) que simplifica la logica del sistema y reduce el acoplamiento entre modulos. La Fig. 5 muestra los estados principales y sus eventos de transicion.

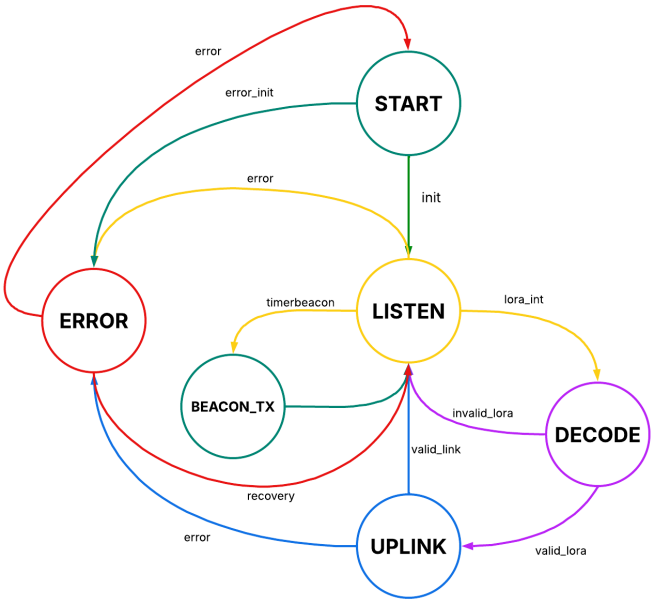


Figura 5: Maquina de estados finita del firmware propio.

La semantica operativa de los estados es la siguiente:

- **START**: inicializa LoRa, WiFi, OLED y el cliente APRS-IS. Si cualquier subsistema critico falla, transiciona a `ERROR`.
- **LISTEN**: mantiene la escucha del canal LoRa, verifica la continuidad del enlace WiFi y vigila el temporizador del beacon.
- **DECODE**: valida la trama recibida, separa `source`, `destination`, `path` y `payload`, y descarta paquetes invalidos.
- **UPLINK**: reformatea el paquete a formato APRS-IS y lo transmite por TCP.
- **BEACON_TX**: genera y envia la posicion del iGate cada 15 minutos.
- **ERROR**: intenta recuperar conectividad WiFi, cliente APRS-IS y subsistema LoRa; si supera el numero maximo de fallos, reinicia el ESP32.

V. Pseudocodigo de alto nivel

El pseudocodigo del Listado 1 resume el comportamiento general del firmware. Se mantiene una separacion entre inicializacion, recepcion, parseo, uplink, beacon y recuperacion.

Listing 1: Pseudocodigo resumido del firmware iGate APRS-IS.

```
CONSTANTES:
    WIFI_SSID, WIFI_PASSWORD
    APRS_SERVER, APRS_PORT
    CALLSIGN, PASSCODE, FILTRO
    LORA_FREQUENCY, LORA_SF, LORA_BW, LORA_CR
    BEACON_INTERVALO, MAX_REINTENTOS

VARIABLES:
    estado <- START
    paquete <- ""
    hay_paquete <- falso
    paquete_valido <- falso
    timer_beacon <- 0
    cont_errores <- 0

SETUP:
    iniciar Serial
    iniciar Display
    estado <- START

LOOP:
    segun estado hacer
        START -> iniciar LoRa, WiFi y APRS-IS
        LISTEN -> escuchar LoRa y revisar timer
        DECODE -> validar y separar campos
        UPLINK -> reformatear a APRS-IS y enviar
        BEACON_TX -> construir posicion y publicar
        ERROR -> intentar recuperacion

START:
    si LoRa.begin falla -> ERROR
    si WiFi no conecta en timeout -> ERROR
    intentar login APRS-IS
    timer_beacon <- tiempo actual
    ir a LISTEN

LISTEN:
    si timer de beacon vence -> BEACON_TX
    si WiFi cae -> ERROR
    si LoRa recibe paquete -> guardar paquete y pasar a DECODE

DECODE:
    limpiar byte 0x3C si existe
    validar presencia de '>' y ':'
    extraer source, dest, path y payload
    si trama valida -> UPLINK
    si no -> LISTEN

UPLINK:
    si socket no esta activo -> reconectar
```

```
    trama_tcp <- source + ">APRS,qAR," + CALLSIGN + ":" + payload + "\r\n"
    si envio falla -> ERROR
    si envio exitoso -> LISTEN

BEACON_TX:
    beacon <- CALLSIGN + ">APRS,TCPIP*:!LAT/LON&COMENTARIO\r\n"
    si APRS-IS esta conectado -> enviar beacon
    reiniciar timer_beacon
    ir a LISTEN

ERROR:
    incrementar cont_errores
    si cont_errores > MAX_REINTENTOS -> reiniciar ESP32
    reintentar WiFi, APRS-IS y LoRa
    si recupera -> LISTEN
```

VI. Tramas de datos

La interaccion del iGate se apoya en cuatro tramas principales: la trama LoRa de entrada, el login APRS-IS, el paquete TCP de salida y el beacon periodico. La Fig. 6 resume su estructura.

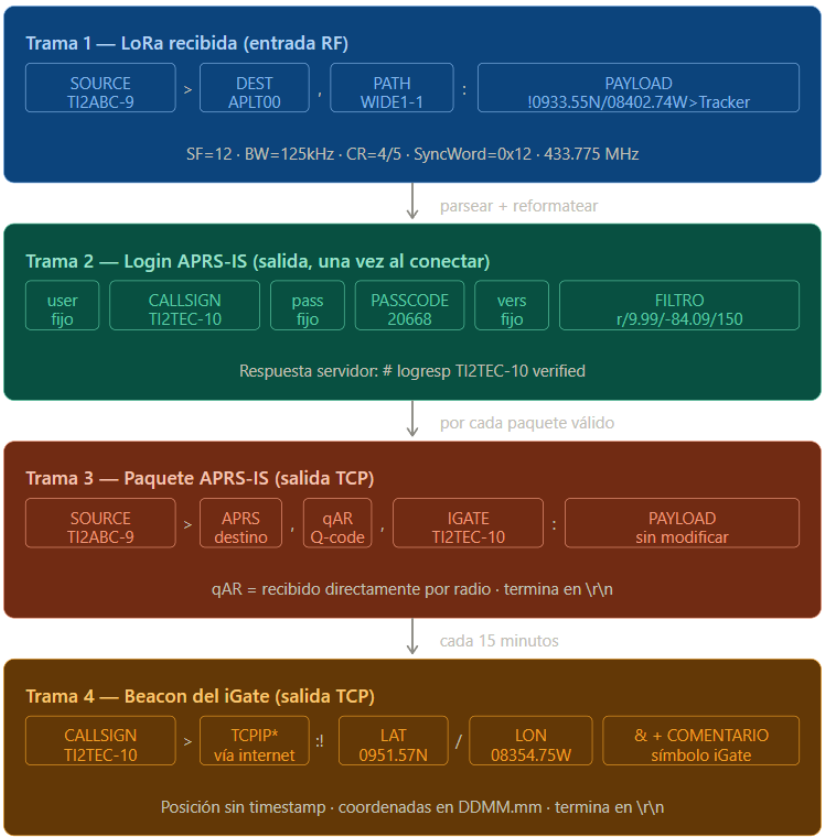


Figura 6: Resumen visual de las tramas de datos manejadas por el firmware.

VI-A. Trama 1: paquete LoRa recibido

El iGate escucha paquetes de texto con formato simplificado tipo AX.25/APRS:

```
SOURCE>DEST,PATH:PAYLOAD
```
(1)

Un ejemplo de entrada es:

```
TI2ABC-9>APLT00,WIDE1-1:!0933.55N/08402.74W>Tracker TEC
```

El parser elimina un byte inicial `0x3C` cuando aparece, valida la presencia de `'>'` y `':'` y rechaza indicativos con longitud fuera del rango de 3 a 9 caracteres.

VI-B. Trama 2: login APRS-IS

Al abrir el socket TCP, el firmware se autentica ante el servidor con la linea:

```
user CALLSIGN pass PASSCODE vers SOFTWARE filter FILTRO\r\n
```
(2)

Este mensaje se envia una sola vez por conexion y la respuesta esperada contiene `verified` o `unverified`.

VI-C. Trama 3: paquete APRS-IS de salida

Por cada paquete LoRa valido, el sistema genera una linea TCP del tipo:

```
SOURCE>APRS,qAR,IGATE:PAYLOAD\r\n
```
(3)

La transformacion conserva el `payload` original y sustituye el destino y la ruta de radio por el bloque `APRS,qAR,IGATE`. El sufijo `qAR` indica que el paquete fue recibido directamente por radio y reenviado por el iGate, sin intermediarios.

VI-D. Trama 4: beacon del iGate

Cada 15 minutos, el firmware publica la posicion del iGate mediante:

```
CALLSIGN>APRS,TCPIP*:!LAT/LON&COMENTARIO\r\n
```
(4)

Las coordenadas se convierten desde grados decimales a formato APRS `DDMM.mmH` para latitud y `DDDMM.mmH` para longitud.

Tabla III: Resumen de direccion y momento de uso de cada trama.

Trama	Direccion	Protocolo	Momento de uso
LoRa raw	Entrada	LoRa RF	Cada paquete recibido desde trackers.
Login APRS-IS	Salida	TCP	Una vez al abrir la conexion con el servidor.
Paquete APRS-IS	Salida	TCP	Por cada paquete LoRa valido.
Beacon del iGate	Salida	TCP	Cada 15 minutos.

VII. Cronograma de trabajo

El cronograma general del proyecto se presenta en la Fig. 7. Para el informe parcial resultan especialmente relevantes las etapas comprendidas entre el diseno del sistema, el diseno de bloques, el diseno del hardware y el desarrollo inicial del firmware. Estas etapas alimentan directamente la presentacion parcial de la semana 9.

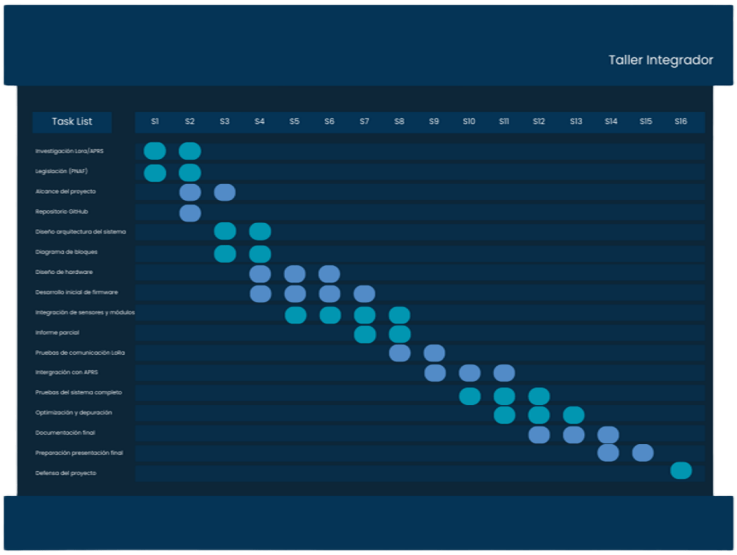


Figura 7: Cronograma general del proyecto elaborado como diagrama tipo Gantt.

Tabla IV: Hitos asociados al informe parcial.

Periodo	Actividad principal	Salida esperada
Semanas 4–5	Validacion de hardware base, organizacion del proyecto y diseno inicial de la arquitectura.	Diagrama de bloques y plan de software.
Semanas 6–8	Desarrollo del firmware propio: WiFi, configuracion LoRa, estructura FSM y documentacion parcial.	Firmware base, pseudocodigo y tramas.
Semana 9	Demostracion en clase del firmware ejecutando en el hardware.	Presentacion parcial.

VIII. Presupuesto estimado

La Tabla V presenta el presupuesto referencial para un prototipo funcional del iGate entregado a la organizacion Rescate Urbano. El presupuesto incluye componentes de hardware y mano de obra profesional de dos ingenieros en electronica a una tarifa de $60 por hora.

Tabla V: Presupuesto referencial del prototipo iGate.

Componente	Cant.	Unitario (USD)	Subtotal (USD)
Hardware			
Tarjeta TTGO LoRa32 T3 v1.6.1	1	28.00	28.00
Antena 433 MHz con conector SMA	1	6.00	6.00
Pantalla OLED SSD1306 (si no viene integrada)	1	5.00	5.00
Bateria LiPo o fuente de alimentacion portatil	1	10.00	10.00
Cable USB y accesorios de conexion	1	4.00	4.00
Material de prototipado (jumpers, headers, soporte)	1	7.00	7.00
Subtotal hardware			**60.00**
Reserva de contingencia hardware (10 %)			6.00
Mano de obra — 2 ingenieros en electronica a $60/hora			
Investigacion y analisis	2 × 8 h	60.00	960.00
Diseno (FSM, diagramas, arquitectura)	2 × 20 h	60.00	2,400.00
Desarrollo del firmware	2 × 30 h	60.00	3,600.00
Pruebas y depuracion	2 × 20 h	60.00	2,400.00
Instalacion y tutoria al personal	2 × 12 h	60.00	1,440.00
Subtotal mano de obra		180 h totales	**10,800.00**
Total estimado del proyecto			**10,866.00**

IX. Avance actual del firmware

A la fecha de este informe, el proyecto cuenta con una base funcional de firmware en PlatformIO. La configuracion del entorno define el framework Arduino, las dependencias `LoRa` (sandeepmistry), `Adafruit SSD1306` y `Adafruit GFX`, con una velocidad de monitor serial de 115200 bps.

En terminos funcionales, el codigo implementa las rutinas requeridas para el corte parcial: inicio de WiFi con tiempo de espera configurable, configuracion explicita del chip LoRa con los parametros de la red LoRa-APRS, recepcion de paquetes con lectura de RSSI y SNR, validacion del formato de trama, construccion de lineas APRS-IS con el sufijo `qAR`, envio de beacon periodico y un estado de recuperacion ante fallos con reinicio automatico del ESP32. El firmware esta listo para las pruebas de integracion de la semana 9 y para la fase posterior de conexion con el servidor local de la clase.

X. Conclusiones

El informe parcial muestra que el firmware del iGate APRS-IS constituye una solucion viable para el problema de visibilidad en operaciones de Rescate Urbano en terreno montañoso. La arquitectura propuesta — iGate en el puesto de comando, trackers con los rescatistas — permite al coordinador monitorear el despliegue del equipo en tiempo real sin depender de infraestructura celular, utilizando unicamente radio LoRa en el enlace de campo e internet en el puesto de comando.

El proyecto cubre los elementos tecnicos requeridos: configuracion de LoRa con parametros adecuados para terreno montañoso (SF=12, alcance estimado 2–5 km), conexion WiFi, cliente APRS-IS, definicion formal de tramas, beacon del iGate y manejo de errores. Como siguiente etapa se plantea consolidar la integracion con el servidor local de la

clase, realizar pruebas de alcance real en campo y documentar los resultados con multiples nodos trackers operando en simultaneo, simulando un despliegue de rescate.

Referencias

[1] Guia de Trabajo: Firmware iGate APRS (Grupos 1-2), documento de curso, Taller Integrador, Instituto Tecnologico de Costa Rica.

[2] LilyGO, "TTGO LoRa32 T3 v1.6.1 Schematic", archivo `T3_V1_6_1.pdf`, repositorio oficial de hardware.

[3] APRS Working Group, "APRS Protocol Reference, Version 1.0", agosto 2000. [En linea]. Disponible: http://www.aprs.org/doc/APRS101.PDF

[4] S. Dimcovic, "APRS-IS Server Protocol". [En linea]. Disponible: http://www.aprs-is.net/Connecting.aspx

[5] S. Mistry, "Arduino LoRa Library". [En linea]. Disponible: https://github.com/sandeepmistry/arduino-LoRa

[6] Codigo fuente del firmware propio: `main.cpp`, `config.h`, `wifi_manager.cpp`, `lora_manager.cpp`, `aprs_parser.cpp`, `aprs_is_client.cpp`, `beacon.cpp` y `display.cpp`.